

①

	P1	P3	P5	P7	P8
2 2	I/O waiting	Writing I/O	Ready	I/O waiting	ready
3 7	ready	ready	Suspended	I/O waiting	executing
4 7	ready	ready	Ready	I/O waiting	exited

②

~~Assuming~~ Assuming on an average page-fault rate although each algorithm works ~~best~~ differently in different scenarios. Therefore ranking relatively.

a) LRU: 2, because it is a ~~best~~ descent algorithm which replaces according to least recently used ~~but~~ but can suffer from page faults in some scenarios. It can also suffer from Belady's anomaly.

b) FIFO: 4, it can suffer from large page faults in many scenarios like simple one. Frame size = 3
Request: 1 2 3 4 1 2 3 4 - - -

c) Optimal: 1, it is perfect algorithm with minimal page faults.

d) Second-chance: 3, It is approximation to LRU.

3

Frans →	1	2	3	4	5	6	7
LRU	20	18	16	10	8	7	7
FIFO	20	18	16	14	10	10	7
Optimal	20	15	11	8	7	7	7

④ a) Yes, it can happen when `foo()` execute `semWait(s)` and `bar()` execute `semWait(r)` simultaneously then `foo()` will be waiting for `semWait(r)` and `bar()` will be waiting for `semWait(s)`. ~~and~~ Therefore they result in being blocked forever.

b) No, because there will come a time when one of them gave a signal (`semSignal`) and other ^{one} will take `semWait`.

Assuming: It can be possible that execution of one process result in some postponement but for the indefinite postponement it has very very less probability (≈ 0). Otherwise it can be possible if remains continue before the other one.

⑤ a) FCFS: $0 + 11 + 63 + 86 + 74 + 55 + 66 + 87 = 442$

b) SSTF: $26 \rightarrow 33 \rightarrow 37 \rightarrow 14 \rightarrow 12 \rightarrow 88 \rightarrow 99 \rightarrow 100$
 $0 + 7 + 4 + 23 + 2 + 76 + 11 + 1 = 124$

c) SCAN: $26 \rightarrow 33 \rightarrow 37 \rightarrow 88 \rightarrow 99 \rightarrow 100 \rightarrow 14 \rightarrow 12$
 $0 + 7 + 4 + 51 + 11 + 1 + 86 + 2 = 162$

d) C-SCAN: $26 \rightarrow 33 \rightarrow 37 \rightarrow 88 \rightarrow 99 \rightarrow 100 \rightarrow 0 \rightarrow 12 \rightarrow 14$
 $0 + 7 + 4 + 51 + 11 + 1 + 100 + 12 + 2 = 188$

Assuming $\text{max} = 100$

⑥ a) Transfer time = time required for transferring the data after access.

$$\begin{aligned} 1 \text{ Revolution will transfer} &= 512 \times 400 \text{ bytes} \\ &= 204800 \text{ bytes} \end{aligned}$$

$$\# \text{ Revolutions required} = \frac{1048576}{204800} = 5.12$$

$$\text{Transfer time} = \frac{5.12}{15000} \times \underbrace{(60 \times 1000)}_{\text{minute to ms}} = 20.48 \text{ ms}$$

$$\text{Total time} = \text{avg. access time} + \text{transfer time} = 26.48 \text{ ms}$$

$$\begin{aligned} \text{b) Average access time} &= \text{Avg. seek time} + \text{Average Rotation latency} \\ &\quad \downarrow \qquad \qquad \qquad \downarrow \\ &\quad 4 \text{ ms} \qquad \qquad \qquad \frac{1}{2} \times \frac{60 \times 1000}{15000} = 2 \text{ ms} \\ &\qquad \qquad \qquad \text{average} \\ &= 6 \text{ ms} \end{aligned}$$

$$\text{c) Rotation delay per rotation} = \frac{60000}{15000} = 4 \text{ ms}$$

For file transfer of 1 MB = 20.48 ms delay is due to ~~file transfer~~ ^{rotation}.

$$\text{d) Time to read one sector} = \frac{1}{400} \times 4 = 0.01 \text{ ms}$$

$$\text{e) Total time to read one track} = 1 \times 4 \text{ ms} = 4 \text{ ms}$$

⑦ 13-direct pointers $\Rightarrow 13 \times 8 \text{ KB} = 104 \text{ KB}$

1 indirect pointer $\Rightarrow$ will point to a block of 8 KB which will have

$$\frac{8 \times 1024}{4} \left(\frac{\text{block size (in bytes)}}{\text{pointer size (in bytes)}} \right)$$

$$= 2 \times 1024 \text{ pointers to } \text{block of } 8 \text{ KB}$$

direct

$$\Rightarrow 16 \text{ MB}$$

1 double indirect pointer: $\frac{8 \times 1024}{4} \times 16 \text{ MB}$

No. of entries
of 1 indirect pointer

One indirect pointer can handle

$$= 32 \text{ GB}$$

1 triple indirect pointer: $\frac{8 \times 1024}{4} \times 32 \text{ GB} = 64 \text{ TB}$

No. of entries of
double indirect pointer

Total size = $64 \text{ TB} + 32 \text{ GB} + 16 \text{ MB} + 104 \text{ KB}$

- ⑧ Solution: Limit the no. of connections to 1 with a single party, can be one of its solution.
2. Record ~~that~~ the challenges that are under process, means sent to ~~an~~ other connections and authentication of them is pending and then if a challenge is received ~~to~~ respond to that challenge only if it is not present in ^{current} ~~the~~ records.
3. If it is possible to receive and send challenges in different ways then do that so that some challenge can't be sent back.
-